Reflection homework 13b

Do Huy Du - 202417108

May 23, 2026

1 Design decisions

I decided to apply the MVC architectural pattern to cleanly separate the three responsibilities of the application. `DateModel` owns the date state and exposes methods to mutate it; `DateView` is a pure Swing UI with no business logic; and `DateController` bridges the two — it registers listeners on the View's buttons and triggers display updates after each model operation.

For date arithmetic I chose `java.time.LocalDate` instead of any manual calculation. This handles all calendar edge cases automatically: leap years, varying month lengths, and boundary shifts such as January 31 moved forward by one month becoming February 28 or 29.

I also capped the input at `MAX_INPUT = 100 000` to prevent absurd date shifts that would be meaningless in practice.

2 Main functionalities

The application displays the current date on startup in a Vietnamese format: *Ngày dd tháng MM năm yyyy*. The user enters a number x in a text field and clicks one of four buttons to move the displayed date forward or backward by x days or x months. Error messages are shown in red below the buttons when the input is invalid, and cleared automatically on the next successful operation.

3 Implementation overview

`DateController.parseInput()` validates the raw string before any model call is made: it rejects empty input, non-numeric strings, negative numbers, and values exceeding the cap, throwing an `IllegalArgumentException` with a descriptive message in each case. After a successful mutation, the controller calls `refreshDisplay()`, which reads the updated `LocalDate` from the model and formats it into the Vietnamese string shown in the View. The three components are wired together in `Main.java` inside

`SwingUtilities.invokeLater` to make the UI not be impacted by MacOS default look and feel